

Quantum AI Digital Money System Training Guide

QuantumAI'nin canlı DCA trading sistemi için uçtan uca kurulum, otomasyon, gözlemlenebilirlik, güvenlik ve saha operasyon rehberi.

Durum **Aktif** Ağ **Mainnet** Çifti **BTCUSDT** DCA Döngüsü **\$25 / 1800s**

Hassasiyet **qty=5 | price=2** SLO Hedefi **%99.95** Belge Sürümü **2025-09-01**

SLO notu: Kullanıcı hedefi "%100 işlemin %99'u başarılı, kalan %1'in %95'i modifiye edilip başarılı" ise toplam başarı oranı $0.99 + 0.01 \times 0.95 = 0.9995$ (%99.95) olur; mimari bu hedefe göre kurgulanmıştır.

Uyarı: Otomatik kripto alım-satım yüksek risk içerir ve kâr garanti edilmez. Gerçek API anahtarlarını yalnızca şifreli saklayın ve demo ortamında sınamadan üretime almayın.

- ▶ 1. Sistem Özeti
- ▶ 2. Mimari & Tasarım
- ▶ 3. Çekirdek Bileşenler
- ▶ 4. Uygulama Ayrıntıları
- ▶ 5. Konfigürasyon
- ▶ 6. Dağıtım
- ▶ 7. Gözlem & Otomasyon
- ▶ 8. Operasyon & Runbook
- ▶ 9. Geliştirme
- ▶ 10. Güvenlik
- ▶ 11. İş Sürekliliği
- ▶ 12. macOS Watcher Kurulumu
- ▶ 13. Tek Seferlik Kurulum Bloğu
- ▶ 14. API & Entegrasyon
- ▶ 15. Destek & Kaynaklar
- ▶ 16. Mega Master Boru Hatları
- ▶ 17. Quantum AI Bot Programları
- ▶ 18. Otomasyon & Sertifika Scriptleri
- ▶ 19. Genişletilmiş Bilgi Notları
- ▶ 0xFF. Ham Ek İçerik Arşivi

1. Sistem Özeti

QAI MegaPipeline, mikro servis mimarisi ve olay güdümlü mesaj kuyrukları ile ölçeklenen, canlı kripto DCA (Dollar Cost Averaging) işlemlerini uçtan uca yöneten kurumsal bir trading çözümüdür. Router, Risk, Executor ve Trade Engine bileşenleri birlikte çalışarak emir yönlendirme, risk kontrolü, yürütme ve sonuç takibini yüksek doğrulukla sağlar.

SLO 99.95%

İdempotent emir üretimi, otomatik retry ve modify/cancel akışları ile hata bütçesi kontrol altında tutulur.

Otonom DCA

Binance spot piyasasında BTCUSDT için 1800 saniyelik periyotlarla 25 USDT tutarında alım yapar; hassasiyet 5/2 basamak.

Çekirdek Teknolojiler

Redis, Redpanda (Kafka API), PostgreSQL, Prometheus, Grafana, FastAPI, Python, Docker Compose, macOS LaunchAgent

Operasyonel Güç

Geniş kapsamlı runbook, otomasyon betikleri, gözlemlenebilirlik panelleri ve kurtarma rutinleri ile saha ekipleri

entegrasyonları.

için hazır hale getirilmiştir.

Temel Yetkinlikler

- Event-driven emir akışı ve gerçek zamanlı risk değerlendirmesi.
- Otomatik ölçeklenme ve yük öngörüsü için ML tabanlı kapasite yönetimi.
- Prometheus ve Grafana ile uçtan uca gözlem ve uyarı altyapısı.
- Dead-letter kuyukları, state snapshot'ları ve hızlı toparlanma mekanizmaları.
- macOS TCC/FDA uyumlu LaunchAgent dosya watcher entegrasyonu.

Operasyon Notları

- Gerçek API anahtarları yalnızca şifreli .env dosyalarında (chmod 600) saklanmalıdır.
- TCC/FDA izinleri verilmeden masaüstü watcher süreçleri çalıştırılmamalıdır.
- Sistem varsayılan olarak Binance mainnet üzerinde çalışır; testnet ve DRY_RUN modları desteklenir.
- Log rotasyonu, metrik toplama ve alarm kanalları haftalık olarak doğrulanmalıdır.

2. Mimari & Tasarım

Mimari, emir yönlendirme ve yürütme katmanlarını yüksek erişilebilirlik ve esneklik sağlayacak şekilde ayrıştırır. Mesaj akışı, Redpanda aracılığıyla asenkron olarak yönetilir; Redis ve PostgreSQL ise durum yönetimi ve kalıcı veri deposu olarak görev yapar.

Bileşen Haritası

```
graph TD
    subgraph Order_Flow [Order Flow]
        A[Router] -->|Small Orders| B[Small Executor]
        A -->|Large Orders| C[Risk Management]
        C -->|Approved Orders| D[Large Executor]
        B --> E[Trade Engine]
    end
```

```

    D --> E
    E -->|Market Orders| F[Exchange APIs]
    G[Grid Bot] -->|Grid Orders| E
end

subgraph Data Services
    H[Market Data Feed]
    I[Position Manager]
    J[Price Aggregator]
end

subgraph Storage
    K[(Redis)]
    L[(PostgreSQL)]
    M[Redpanda/Kafka]
end

H --> J
J --> E
E --> I
I --> K
I --> L
A --> M
M --> B
M --> D

```

Dağıtım Topolojisi

```

graph TD
    subgraph Load Balancer
        LB[NGINX]
    end

    subgraph Application Tier
        A1[API Server 1]
        A2[API Server 2]
        R1[Router 1]
        R2[Router 2]
    end

    subgraph Processing Tier
        E1[Executor 1]
        E2[Executor 2]
        T1[Trade Engine 1]
    end

```

```

    T2[Trade Engine 2]
end

subgraph Data Tier
    D1[(Redis Primary)]
    D2[(Redis Replica)]
    D3[(PostgreSQL Primary)]
    D4[(PostgreSQL Replica)]
    K1[Kafka Broker 1]
    K2[Kafka Broker 2]
end

LB --> A1
LB --> A2
A1 --> R1
A2 --> R2
R1 --> K1
R2 --> K2
K1 --> E1
K2 --> E2
E1 --> T1
E2 --> T2
T1 --> D1
T2 --> D1
D1 --> D2
T1 --> D3
T2 --> D3
D3 --> D4

```

Veri Akışı

```

sequenceDiagram
    participant C as Client
    participant A as API
    participant R as Router
    participant K as Kafka
    participant E as Executor
    participant T as Trade Engine
    participant D as Database

    C->>A: Submit Order
    A->>R: Route Order
    R->>K: Publish Order Event
    K->>E: Consume Order

```

E->>T: Execute Order
T->>D: Store Result
T->>K: Publish Execution Event
K->>A: Consume Execution
A->>C: Send Response

Mesaj Konuları ve Hatlar

- `orders.incoming` : Yeni emirlerin giriş kuyruğu.
- `route.slices` : Router sonrası parçalanmış emirler.
- `orders.executions` : Yürütülmüş emir sonuçları.
- `tick.shibusdt` : Piyasa veri akışı.
- `sim.requests` ve `sim.results` : Simülasyon giriş-çıkışları.

3. Çekirdek Bileşenler

Router Service

- Emir büyüklüğüne göre küçük/büyük hat ayrımı.
- Gerçek zamanlı sıra yönetimi ve load balancing.
- Redpanda aracılığıyla kuyruğa yazma ve takip.
- Idempotent routing anahtarları.

Risk Management

- Portföy maruziyet ve limit kontrolü.
- Value at Risk (VaR) hesapları.
- Gerçek zamanlı risk değerlendirmesi ve ret gerekçeleri.
- Audit logging ve post-trade raporlama.

Order Executors

- Küçük emir agregasyonu ve

Trade Engine

- Order book yönetimi ve

mikro lot yönetimi.

- Büyük emirler için TWAP/VWAP stratejileri.
- Piyasa etkisi minimizasyonu.
- Failover ve retry stratejileri.

eşleştirme.

- Pazar entegrasyonu ve fiyat-zaman önceliği.
- En iyi yürütme politikaları.
- Latency ölçümü ve SLA takibi.

Grid Bot

- Otomatik grid stratejileri ve dinamik aralık ayarı.
- Piyasa koşullarına göre yeniden dengelenme.
- Risk limitleri ile entegre emir akışı.

Auto-Scaling System

- CPU, bellek, istek ve hata oranı metriklerinin toplanması.
- ML tabanlı yük tahmini ve anomali tespiti.
- Soğuma süreleri ve kademeli ölçeklenme politikaları.

Disaster Recovery

- PostgreSQL, Redis ve Kafka için zamanlanmış yedekler.
- Konfigürasyon ve gizli anahtar yedekleme.
- Otomatik geri yükleme ve sağlık doğrulama akışları.

4. Uygulama Ayrıntıları

Mesaj Akışı

sequenceDiagram

```
participant Client
participant Router
participant Risk
participant Executor
participant TradeEngine
```

```
Client->>Router: Submit Order
Router->>Risk: Large Order
Risk->>Executor: Approved Order
Executor->>TradeEngine: Execution Request
TradeEngine->>Executor: Execution Response
Executor->>Client: Order Status
```

Durum Yönetimi (Redis)

```
# Router State
router_state:
  active_orders: Hash
    order_id: order_details
  order_queues: SortedSet
    queue_id: priority_score

# Risk State
risk_state:
  positions: Hash
    symbol: position_details
  risk_limits: Hash
    symbol: limit_details

# Trade Engine State
trade_engine_state:
  order_books: Hash
    symbol: book_snapshot
  active_orders: Hash
    order_id: order_details
```

Veri Kalıcılığı

- **Redis:** Anlık emir durumu, risk metriği, piyasa verisi önbellesi.
- **PostgreSQL:** Tamamlanan emirler, işlem geçmişi, pozisyon snapshot'ları.
- **Redpanda:** Emir olayları, yürütme akışları ve simülasyon kuyrukları.

Hata Yönetimi ve Kurtarma

```
from functools import wraps
import time
from typing import Any, Callable
import redis
from tenacity import retry, stop_after_attempt, wait_exponential

class CircuitBreaker:
    def __init__(self, failure_threshold: int = 5, reset_timeout: i
        self.failure_count = 0
        self.failure_threshold = failure_threshold
        self.reset_timeout = reset_timeout
        self.last_failure_time = 0
        self.is_open = False

    def __call__(self, func: Callable) -> Callable:
        @wraps(func)
        def wrapper(*args: Any, **kwargs: Any) -> Any:
            if self.is_open:
                if time.time() - self.last_failure_time > self.rese
                    self.is_open = False
                    self.failure_count = 0
            else:
                raise Exception("Circuit breaker is open")

            try:
                result = func(*args, **kwargs)
                self.failure_count = 0
                return result
            except Exception as e:
                self.failure_count += 1
                self.last_failure_time = time.time()
                if self.failure_count >= self.failure_threshold:
                    self.is_open = True
                raise e

        return wrapper
```

```

@retry(stop=stop_after_attempt(3), wait=wait_exponential(multiplier
def execute_order(order_id: str) -> dict:
    try:
        result = trade_engine.execute(order_id)
        return result
    except ConnectionError as e:
        logger.error(f"Connection error executing order {order_id}:
        raise
    except Exception as e:
        logger.error(f"Unrecoverable error executing order {order_i
        raise

```

Dead Letter Queue İşleyicisi

```

import json

class DeadLetterQueue:
    def __init__(self, redis_client: redis.Redis):
        self.redis = redis_client
        self.dlq_key = "orders:dlq"

    def push(self, order: dict, error: Exception) -> None:
        entry = {
            "order": order,
            "error": str(error),
            "timestamp": time.time(),
            "retries": 0
        }
        self.redis.rpush(self.dlq_key, json.dumps(entry))

    def process_dlq(self) -> None:
        while True:
            entry = self.redis.lpop(self.dlq_key)
            if not entry:
                break

            entry = json.loads(entry)
            if entry["retries"] < 3:
                try:
                    execute_order(entry["order"]["id"])
                except Exception as e:

```

```

        entry["retries"] += 1
        entry["last_error"] = str(e)
        self.redis.rpush(self.dlq_key, json.dumps(entry))
    else:
        logger.error(f"Order permanently failed: {entry}")

```

Durum Kurtarma

```

from dataclasses import dataclass, asdict
from typing import Dict
import json
import time

@dataclass
class OrderState:
    order_id: str
    status: str
    details: dict
    version: int

class StateManager:
    def __init__(self, redis_client: redis.Redis):
        self.redis = redis_client
        self.snapshot_key = "state:snapshot"
        self.changelog_key = "state:changelog"

    def save_snapshot(self, state: Dict[str, OrderState]) -> None:
        snapshot = {
            "timestamp": time.time(),
            "state": {k: asdict(v) for k, v in state.items()}
        }
        self.redis.set(self.snapshot_key, json.dumps(snapshot))

    def append_changelog(self, event: dict) -> None:
        self.redis.rpush(self.changelog_key, json.dumps(event))

    def recover_state(self) -> Dict[str, OrderState]:
        snapshot = self.redis.get(self.snapshot_key)
        if not snapshot:
            return {}

        snapshot = json.loads(snapshot)

```

```

state = {k: OrderState(**v) for k, v in snapshot["state"].items()}
snapshot_time = snapshot["timestamp"]

changelog = self.redis.lrange(self.changelog_key, 0, -1)
for event_json in changelog:
    event = json.loads(event_json)
    if event["timestamp"] > snapshot_time:
        self._apply_event(state, event)
return state

def _apply_event(self, state: Dict[str, OrderState], event: dict):
    order_id = event["order_id"]
    if event["type"] == "ORDER_CREATED":
        state[order_id] = OrderState(
            order_id=order_id,
            status="created",
            details=event["details"],
            version=1
        )
    elif event["type"] == "ORDER_UPDATED" and order_id in state:
        state[order_id].status = event["status"]
        state[order_id].details.update(event["details"])
        state[order_id].version += 1

```

Performans Optimizasyonu

```

from functools import lru_cache
from typing import Optional, List, Dict
from datetime import timedelta
import json
import redis
from redis_cache import RedisCache

class MarketDataCache:
    def __init__(self, redis_client: redis.Redis):
        self.redis = redis_client
        self.local_cache = RedisCache(redis_client)
        self.orderbook_key = "market:orderbook:{symbol}"
        self.ticker_key = "market:ticker:{symbol}"

    @lru_cache(maxsize=1000)
    def get_last_price(self, symbol: str) -> float:

```

```

        return float(self.redis.get(f"price:{symbol}") or 0)

    @local_cache.cache(ttl=timedelta(seconds=5))
    def get_orderbook(self, symbol: str) -> dict:
        key = self.orderbook_key.format(symbol=symbol)
        data = self.redis.get(key)
        return json.loads(data) if data else {}

    def invalidate_cache(self, symbol: str) -> None:
        self.get_last_price.cache_clear()
        self.local_cache.invalidate(f"market:orderbook:{symbol}")

from typing import List, Dict
import asyncio
import json
from collections import defaultdict

class OrderBatchProcessor:
    def __init__(self, batch_size: int = 100, timeout: float = 0.5):
        self.batch_size = batch_size
        self.timeout = timeout
        self.orders: defaultdict = defaultdict(list)
        self.processing = False

    async def add_order(self, symbol: str, order: dict) -> None:
        self.orders[symbol].append(order)
        if len(self.orders[symbol]) >= self.batch_size and not self:
            await self.process_batches()

    async def process_batches(self) -> None:
        self.processing = True
        try:
            for symbol, orders in self.orders.items():
                if not orders:
                    continue
                chunks = [orders[i:i + self.batch_size] for i in range(0, len(orders), self.batch_size)]
                for chunk in chunks:
                    try:
                        await self._process_chunk(symbol, chunk)
                    except Exception as e:
                        logger.error(f"Error processing chunk: {e}")
                        await self._handle_failed_chunk(symbol, chunk)
            self.orders.clear()
        finally:
            self.processing = False

```

```

        self.orders[symbol] = []
    finally:
        self.processing = False

    async def _process_chunk(self, symbol: str, orders: List[dict])
        aggregated = defaultdict(float)
        for order in orders:
            aggregated[order["price"]] += order["quantity"]
        for price, quantity in aggregated.items():
            await trade_engine.execute_batch_order(symbol, price, q

    async def _handle_failed_chunk(self, symbol: str, orders: List[
        for order in orders:
            await dead_letter_queue.push(order)

from typing import List
import aiohttp
import asyncio
from dataclasses import dataclass

@dataclass
class ServiceNode:
    host: str
    port: int
    weight: int = 100
    active: bool = True

class LoadBalancer:
    def __init__(self):
        self.nodes: List[ServiceNode] = []
        self.current_index = 0
        self.lock = asyncio.Lock()

    def add_node(self, node: ServiceNode) -> None:
        self.nodes.append(node)

    def remove_node(self, host: str, port: int) -> None:
        self.nodes = [n for n in self.nodes if not (n.host == host

    async def get_next_node(self) -> ServiceNode:
        async with self.lock:
            if not self.nodes:

```

```

        raise Exception("No available nodes")
    while True:
        self.current_index = (self.current_index + 1) % len
        node = self.nodes[self.current_index]
        if node.active:
            return node

    async def health_check(self) -> None:
        while True:
            for node in self.nodes:
                try:
                    async with aiohttp.ClientSession() as session:
                        url = f"http://{node.host}:{node.port}/heal
                    async with session.get(url) as response:
                        node.active = response.status == 200
                except Exception:
                    node.active = False
            await asyncio.sleep(30)

```

5. Konfigürasyon

.env.local Konumları

- /Volumes/LaCie/Container-QuantumAI/QuantumAI-Dockerized-System/.env.local
- /Users/erenuludemir/QuantumAI-Dockerized-System/.env.local

Çekirdek Servis Ortamı

```

# Core Services
ROUTER_PORT=8081
RISK_PORT=8082
SMALL_EXEC_PORT=8083
LARGE_EXEC_PORT=8084
TRADE_ENGINE_PORT=8085
GRID_BOT_PORT=8086

```

```
# Logging
LOG_LEVEL=INFO
LOG_FORMAT=json

# Dependencies
REDIS_URL=redis://redis:6379
KAFKA_BROKERS=redpanda:9092
```

Üretim DCA Parametreleri

```
SYMBOL=BTCUSDT
DCA_AMOUNT=25
DCA_INTERVAL=1800
BINANCE_TESTNET=0
DRY_RUN=0

QUANTITY_PRECISION=5
PRICE_PRECISION=2
MIN_NOTIONAL_USDT=10
ROUNDING_MODE=down
AUTO_FETCH_FILTERS=1

RUN_DEV=0
BINANCE_BOT_PORT=8080
FLASK_RUN_PORT=5000
FLASK_DEBUG=0
FLASK_ENV=production

VITE_NODE_API=http://localhost:8080
VITE_PY_API=http://localhost:8000
WS_BROADCAST_URI=ws://localhost:8765

BINANCE_API_KEY=***REDACTED***
BINANCE_SECRET_KEY=***REDACTED***
BINANCE_API_SECRET=***REDACTED***

LOG_LEVEL=INFO
LOG_DIR=/root/.quantumai/logs
METRICS_ENABLED=1
ALERT_EMAIL=ops@example.com
ALERT_WEBHOOK=
```



```
ORDER_DEDUP_TTL=3600
MAX_RETRY=5
RETRY_BACKOFF=2
```

Auto-Scaling Konfigürasyonu

```
{
  "metrics": {
    "cpu_usage": {
      "query": "rate(process_cpu_seconds_total[5m])",
      "threshold": 0.7
    },
    "memory_usage": {
      "query": "process_resident_memory_bytes",
      "threshold": 0.8
    },
    "request_rate": {
      "query": "rate(http_requests_total[5m])",
      "threshold": 1000
    },
    "latency": {
      "query": "rate(http_request_duration_seconds_sum[5m])",
      "threshold": 0.5
    }
  },
  "scaling": {
    "check_interval": 300,
    "cooldown_period": 600,
    "prediction_window": 3600,
    "ml_model": {
      "type": "random_forest",
      "training_period": "7d",
      "features": ["cpu", "memory", "requests", "latency"]
    }
  },
  "services": {
    "trade_engine": {
      "min_replicas": 2,
      "max_replicas": 10,
      "target_cpu_utilization": 0.7
    },
  },
}
```

```

    "risk_service": {
      "min_replicas": 1,
      "max_replicas": 5,
      "target_memory_utilization": 0.8
    }
  }
}

```

Disaster Recovery Ayarları

```

{
  "backup": {
    "schedule": {
      "database": "0 */6 * * *",
      "redis": "0 * * * *",
      "kafka": "0 */12 * * *"
    },
    "retention": {
      "database": "7d",
      "redis": "24h",
      "kafka": "3d"
    },
    "compression": true
  },
  "storage": {
    "type": "s3",
    "bucket": "qai-backups",
    "region": "us-east-1",
    "path_prefix": "prod/"
  },
  "recovery": {
    "healthcheck_timeout": 300,
    "restore_order": ["database", "redis", "kafka", "services"]
    "verification": {
      "run_tests": true,
      "check_data_integrity": true
    }
  }
}

```

- Redis: Durum, kuyruk ve idempotency anahtarları.
- Redpanda: Yüksek hacimli mesajlaşma ve olay akışı.
- PostgreSQL: Kalıcı trading ve raporlama verileri.
- Prometheus & Grafana: Metrik ve dashboard altyapısı.

6. Dağıtım

Docker Compose (Çekirdek Hizmetler)

```
version: '3.8'

services:
  redis:
    image: redis:latest
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data

  redpanda:
    image: vectorized/redpanda:latest
    ports:
      - "9092:9092"
    command:
      - redpanda
      - start
      - --smp 1
      - --memory 1G

  router:
    image: qai/router-service:latest
    ports:
      - "8081:8081"
    environment:
      - ROUTER_PORT=8081
      - LOG_LEVEL=INFO
```

```
risk:
  image: qai/risk-service:latest
  ports:
    - "8082:8082"
  environment:
    - RISK_PORT=8082
    - LOG_LEVEL=INFO

small_executor:
  image: qai/small-executor:latest
  ports:
    - "8083:8083"
  environment:
    - SMALL_EXEC_PORT=8083
    - LOG_LEVEL=INFO

large_executor:
  image: qai/large-executor:latest
  ports:
    - "8084:8084"
  environment:
    - LARGE_EXEC_PORT=8084
    - LOG_LEVEL=INFO

trade_engine:
  image: qai/trade-engine:latest
  ports:
    - "8085:8085"
  environment:
    - TRADE_ENGINE_PORT=8085
    - LOG_LEVEL=INFO

grid_bot:
  image: qai/grid-bot:latest
  ports:
    - "8086:8086"
  environment:
    - GRID_BOT_PORT=8086
    - LOG_LEVEL=INFO

volumes:
  redis_data:
```

Çalıştırma Akışı

```
cd /Volumes/LaCie/Container-QuantumAI/QuantumAI-Dockerized-System

docker compose up -d --build

curl -fsS http://127.0.0.1:8080/health

docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Uygulama Giriş Noktası

```
CMD ["unicorn", "-w", "1", "-k", "gthread", "-t", "120", "-b", "0.0.0.0:80
```

Development modu: `RUN_DEV=1` ile Flask debug sunucusu 5000 portunda ayağa kalkar.

7. Gözlem & Otomasyon

ML Tabanlı İzleme

Predictive Analytics

- Kaynak kullanım tahmini ve kapasite optimizasyonu.
- Anomali tespiti ve trend analizi.
- Model doğrulama ve sürekli yeniden eğitim.

Capacity Planning

- Trafik desen analizi ve ölçek eşikleri.
- Maliyet optimizasyonu ve kaynak verimliliği.

Otomasyon Örnekleri

```

def check_scaling_triggers():
    metrics = collect_system_metrics()
    prediction = ml_model.predict(metrics)

    if prediction.load_factor > SCALE_UP_THRESHOLD:
        scale_service("trade_engine", increase_by=2)
    elif prediction.load_factor < SCALE_DOWN_THRESHOLD:
        scale_service("trade_engine", decrease_by=1)

async def execute_recovery_plan():
    await stop_services(affected_services)
    await restore_database()
    await restore_redis_data()
    await restore_kafka_topics()
    await start_services(affected_services)
    health_status = await verify_system_health()
    return health_status

```

Prometheus Metrik Toplayıcı

```

from prometheus_client import Counter, Histogram, Gauge
import time
from functools import wraps

class MetricsCollector:
    def __init__(self):
        self.order_counter = Counter(
            'qai_orders_total',
            'Total number of orders processed',
            ['service', 'type', 'status']
        )
        self.order_latency = Histogram(
            'qai_order_latency_seconds',
            'Order processing latency',
            ['service', 'stage'],
            buckets=[0.1, 0.5, 1.0, 2.0, 5.0]
        )
        self.memory_gauge = Gauge(
            'qai_memory_usage_bytes',
            'Current memory usage',

```

```

        ['service']
    )
    self.cpu_gauge = Gauge(
        'gai_cpu_usage_percent',
        'Current CPU usage',
        ['service']
    )
    self.error_counter = Counter(
        'gai_errors_total',
        'Total number of errors',
        ['service', 'type']
    )
    self.queue_gauge = Gauge(
        'gai_queue_depth',
        'Current queue depth',
        ['service', 'queue']
    )

def track_latency(self, service: str, stage: str):
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, **kwargs):
            start_time = time.time()
            try:
                result = await func(*args, **kwargs)
                self.order_latency.labels(service=service, stag
                return result
            except Exception as e:
                self.error_counter.labels(service=service, type
                raise
            return wrapper
        return decorator

```

Prometheus & Grafana Konfigürasyonu

```

global:
    scrape_interval: 15s
    evaluation_interval: 15s

scrape_configs:
    - job_name: 'gai_services'
      static_configs:

```

```

    - targets:
      - 'router:8081'
      - 'risk:8082'
      - 'small_executor:8083'
      - 'large_executor:8084'
      - 'trade_engine:8085'
      - 'grid_bot:8086'
    metrics_path: '/metrics'
    scrape_interval: 5s

- job_name: 'node_exporter'
  static_configs:
    - targets: ['node_exporter:9100']

- job_name: 'redis_exporter'
  static_configs:
    - targets: ['redis_exporter:9121']

rule_files:
  - 'alert_rules.yml'

{
  "dashboard": {
    "title": "QAI Trading System Overview",
    "panels": [
      {
        "title": "Order Processing Rate",
        "type": "graph",
        "targets": [
          {"expr": "rate(qai_orders_total[5m])", "legendFormat": "{
        ]
      },
      {
        "title": "Order Latency",
        "type": "heatmap",
        "targets": [
          {"expr": "rate(qai_order_latency_seconds_bucket[5m])", "l
        ]
      },
      {
        "title": "Error Rate",
        "type": "graph",

```



```

        "targets": [
            {"expr": "rate(qai_errors_total[5m])", "legendFormat": "{
        ],
    },
    {
        "title": "Queue Depth",
        "type": "gauge",
        "targets": [
            {"expr": "qai_queue_depth", "legendFormat": "{{service}}
        ]
    }
]
}
}

```

Health Check Servisi

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import aioredis
import psycopg2
from kafka import KafkaAdminClient

class HealthStatus(BaseModel):
    status: str
    details: dict

class HealthChecker:
    def __init__(self, app: FastAPI):
        self.app = app
        self.redis = aioredis.from_url("redis://localhost:6379")
        self.kafka = KafkaAdminClient(bootstrap_servers='localhost:

@app.get("/health", response_model=HealthStatus)
async def health_check():
    try:
        await self.check_redis()
        self.check_kafka()
        self.check_database()
        return HealthStatus(
            status="healthy",
            details={

```

```
        "redis": "connected",
        "kafka": "connected",
        "database": "connected"
    }
)
except Exception as e:
    raise HTTPException(status_code=503, detail=str(e))

async def check_redis(self):
    await self.redis.ping()

def check_kafka(self):
    self.kafka.list_topics()

def check_database(self):
    with psycopg2.connect(dbname="trading", user="trader", pass
        with conn.cursor() as cur:
            cur.execute("SELECT 1")
```

8. Operasyon & Runbook

Sistem İşlemleri

```
./scripts/start.sh
./scripts/stop.sh
./scripts/health.sh
./scripts/scaling/run_auto_scaler.sh
```

İzleme ve Bakım

```
./scripts/monitor.sh metrics
./scripts/monitor.sh predictions
./scripts/monitor.sh scaling

tail -f /var/log/qai/auto_scaler.log
```

Yedekleme & Kurtarma

```
./scripts/dr/run_backup.sh
./scripts/dr/list_backups.sh
./scripts/dr/run_restore.sh /path/to/backup
./scripts/dr/test_dr.sh
```

Runbook Tablosu

Belirti	Neden	Çözüm
<code>APIError -1013:</code> <code>LOT_SIZE</code>	Miktar adımı hassasiyetini aşma	Qty değerini 5 ondalık basamağa aşağı yuvarla
Dev server uyarısı	Flask dev sunucu production'da	Gunicorn kullan, <code>RUN_DEV=0</code> ayarla
8080 port çakışması	Başka servis portu kapmış durumda	Çakışan servisi durdur, yalnız Gunicorn'u açık bırak
Health = "unhealthy"	Uygulama ayağa kalkmadı /health dönmüyor	Logları incele, env ve port eşleşmesini doğrula
TCC: Operation not permitted	Full Disk Access verilmemiş	macOS TCC ayarlarını güncelle, logda Denied kalkmalı

Sorun Giderme Komutları

```
./scripts/health.sh

docker compose logs -f [service_name]

docker compose exec redpanda rpk topic list

docker compose exec redpanda rpk group list

docker compose exec redis redis-cli info memory

docker stats
```

Diagnostik Sınıfı

```
import asyncio
import aiohttp
import redis
import psycopg2
import json
from kafka import KafkaAdminClient

class SystemDiagnostics:
    def __init__(self):
        self.services = {
            'api': 'http://localhost:8000',
            'router': 'http://localhost:8081',
            'risk': 'http://localhost:8082',
            'executor': 'http://localhost:8083'
        }
        self.kafka_topics = [
            'orders.incoming',
            'orders.routed',
            'route.slices',
            'orders.executions'
        ]

    async def check_service_health(self):
        results = {}
        async with aiohttp.ClientSession() as session:
            for service, url in self.services.items():
                try:
                    async with session.get(f"{url}/health") as resp:
                        results[service] = "healthy" if response.status == 200 else "unhealthy"
                except Exception as e:
                    results[service] = f"error: {str(e)}"
        return results

    def check_kafka_topics(self):
        client = KafkaAdminClient(bootstrap_servers='localhost:9092')
        status = {}
        for topic in self.kafka_topics:
            try:
                metadata = client.describe_topics([topic])[0]
                status[topic] = "OK"
            except Exception as e:
                status[topic] = f"error: {str(e)}"
```

```

        status[topic] = {
            'partitions': len(metadata.partitions),
            'replicas': len(metadata.partitions[0].replicas
            'in_sync': len(metadata.partitions[0].isr)
        }
    except Exception as e:
        status[topic] = {'error': str(e)}
    return status

def check_database_connections(self):
    results = {}
    try:
        with psycopg2.connect(dbname="qaidb", user="qai", passw
            with conn.cursor() as cur:
                cur.execute("SELECT COUNT(*) FROM orders")
                results['postgres'] = "connected"
    except Exception as e:
        results['postgres'] = f"error: {str(e)}"

    try:
        r = redis.Redis(host='localhost', port=6379, db=0)
        r.ping()
        results['redis'] = "connected"
    except Exception as e:
        results['redis'] = f"error: {str(e)}"

    return results

```

Diagnostik Çalıştırma

```

import asyncio
import json

async def run_health_check():
    diag = SystemDiagnostics()
    health = await diag.check_service_health()
    print("Service Health:", json.dumps(health, indent=2))
    topics = diag.check_kafka_topics()
    print("Kafka Topics:", json.dumps(topics, indent=2))
    dbs = diag.check_database_connections()
    print("Databases:", json.dumps(dbs, indent=2))

```

```
if __name__ == "__main__":  
    asyncio.run(run_health_check())
```

Kurtarma Prosedürleri

```
./scripts/stop.sh  
./scripts/start.sh
```

```
docker compose stop [service]  
docker compose rm [service]  
docker compose up -d [service]
```

```
./scripts/backup.sh  
./scripts/restore.sh [backup_date]  
./scripts/verify_state.sh
```

```
docker compose exec redpanda rpk group reset [group_id]  
./scripts/recreate-topics.sh  
./scripts/verify_messages.sh
```

9. Geliştirme

Yerel Ortam Kurulumu

```
git clone https://github.com/quantum-ai/trading-system.git  
cd trading-system
```

```
docker compose up -d redis redpanda
```

```
export LOG_LEVEL=DEBUG  
export REDIS_URL=redis://localhost:6379  
export KAFKA_BROKERS=localhost:9092
```

```
./scripts/start.sh
```

Test Planı

```
pytest services/*/tests/unit/  
pytest tests/integration/  
./scripts/load_test.sh all
```

- Unit: miktar/price rounding, idempotency anahtarları.
- Entegrasyon: Binance testnet ile DRY_RUN=1 doğrulaması.
- Canlı prova: minimum notional ile tek döngü.
- Geri kazanım: Hata simülasyonu sonrası retry/modifiye akışı.

Sistem Gereksinimleri

- CPU: 4+ çekirdek.
- RAM: 16 GB ve üzeri.
- Disk: 100 GB boş alan.
- OS: macOS 12+, Linux veya WSL2.
- Ağ: Sabit ve düşük gecikmeli bağlantı.

İzleme

- Log: `docker compose logs -f [service]`
- Health: `./scripts/health.sh`
- Simülasyon: `./scripts/tail_results.sh`
- Metrikler: Grafana `http://localhost:3000`

10. Güvenlik

Ağ ve İletişim Güvenliği

```
from cryptography import x509  
from cryptography.hazmat.primitives import hashes
```

```

import ssl
import aiohttp

class SecureCommunication:
    def __init__(self):
        self.ssl_context = self._create_ssl_context()
        self.certificate_pins = self._load_certificate_pins()

    def _create_ssl_context(self) -> ssl.SSLContext:
        context = ssl.create_default_context()
        context.minimum_version = ssl.TLSVersion.TLSv1_3
        context.set_ciphers('TLS_AES_256_GCM_SHA384:TLS_CHACHA20_PO
        return context

    async def verify_certificate_pin(self, cert_der: bytes) -> bool:
        cert_hash = hashes.Hash(hashes.SHA256())
        cert_hash.update(cert_der)
        digest = cert_hash.finalize()
        return digest.hex() in self.certificate_pins

    async def secure_request(self, url: str, method: str, data: dict):
        async with aiohttp.ClientSession() as session:
            async with session.request(method, url, ssl=self.ssl_co
            cert = response.connection.transport.get_extra_info
            if not await self.verify_certificate_pin(cert):
                raise SecurityError("Certificate pin verificati
            return await response.json()

```

Erişim Kontrolü ve Kimlik Doğrulama

```

import jwt
from datetime import datetime, timedelta
from dataclasses import dataclass
from argon2 import PasswordHasher

@dataclass
class User:
    id: str
    username: str
    roles: list
    password_hash: str
    mfa_secret: str | None

```



```

class SecurityManager:
    def __init__(self, secret_key: str):
        self.secret_key = secret_key
        self.ph = PasswordHasher()
        self.max_attempts = 5
        self.lockout_duration = timedelta(minutes=15)
        self._failed_attempts = {}

    def create_jwt_token(self, user: User) -> str:
        payload = {
            'sub': user.id,
            'roles': user.roles,
            'exp': datetime.utcnow() + timedelta(hours=1)
        }
        return jwt.encode(payload, self.secret_key, algorithm='HS256')

    def verify_password(self, username: str, password: str) -> bool:
        if self._is_account_locked(username):
            raise SecurityError("Account is temporarily locked")
        try:
            user = self._get_user(username)
            if self.ph.verify(user.password_hash, password):
                self._reset_failed_attempts(username)
                return True
            self._record_failed_attempt(username)
            return False
        except Exception:
            self._record_failed_attempt(username)
            return False

    def verify_mfa_code(self, user: User, code: str) -> bool:
        if not user.mfa_secret:
            return False
        return pyotp.TOTP(user.mfa_secret).verify(code)

```

Veri Güvenliği

```

from cryptography.fernet import Fernet

class DataEncryption:
    def __init__(self, encryption_key: bytes):

```

```
self.fernet = Fernet(encryption_key)
self.sensitive_fields = {'api_key', 'secret_key', 'password'}

def encrypt_sensitive_data(self, data: dict) -> dict:
    result = data.copy()
    for field in self.sensitive_fields:
        if field in result:
            result[field] = self.fernet.encrypt(result[field].e
    return result

def decrypt_sensitive_data(self, data: dict) -> dict:
    result = data.copy()
    for field in self.sensitive_fields:
        if field in result:
            result[field] = self.fernet.decrypt(result[field].e
    return result
```

Politikalar

- API anahtarları gizli kasa veya KMS üzerinde tutulur.
- RBAC ile rol bazlı yetkilendirme ve audit logging etkinleştirilmiştir.
- MFA, rate limiting ve IP allowlist politikaları uygulanır.
- Düzenli güvenlik testleri ve bağımsız denetimler planlanır.

11. İş Sürekliliği

Yedekleme Takvimi

- Veritabanı: 6 saatte bir tam snapshot.
- Redis: Saatlik snapshot ve opsiyonel AOF.
- Kafka: 12 saatte bir topic arşivi.
- Konfigürasyon: Günlük snapshot.

Geri Yükleme Adımları

1. Tüm servisleri durdur.

2. Son yedeği yükle (database > redis > kafka > services).
3. Veri bütünlüğünü doğrula ve servisleri başlat.
4. Sağlık kontrolleri ve regresyon testleri çalıştır.

Failover Süreci

1. Health endpoint'lerini izle ve sapmaları tespit et.
2. Degradasyon durumunda yedek instance'a geç.
3. Post-failover doğrulaması yap ve kök nedeni araştır.

12. macOS Watcher Kurulumu

Bu bölüm desktop watcher otomasyonunun FDA uyumlu kurulumu için gereken adımları içerir. Amaç, masaüstüne düşen dosyaları güvenli biçimde

~/ .qai_optimize/inbox/ dizinine almak ve Docker tabanlı iş akışını tetiklemektir.

LaunchAgent Tanımı

```
~/Library/LaunchAgents/com.qai.desktopwatcher.plist
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.ap
<plist version="1.0">
<dict>
  <key>Label</key><string>com.qai.desktopwatcher</string>
  <key>ProgramArguments</key>
  <array>
    <string>/Users/erenuludemir/.qai_optimize/bin/qai_desktop_watch
  </array>
  <key>WatchPaths</key><array><string>/Users/erenuludemir/Desktop</
  <key>RunAtLoad</key><true/>
  <key>KeepAlive</key><true/>
  <key>StandardOutPath</key><string>/Users/erenuludemir/.qai_optimi
  <key>StandardErrorPath</key><string>/Users/erenuludemir/.qai_opti
</dict>
</plist>
```

TCC / Full Disk Access

- System Settings > Privacy & Security > Full Disk Access menüsünden
/bin/bash , /usr/bin/env , /usr/bin/ditto , /usr/bin/rsync ,
/usr/bin/find uygulamalarını yetkilendir.
- Terminal/iTerm için Files and Folders altında Desktop erişimini aç.
- Doğrulama için: `log stream --style compact --predicate 'subsystem == "com.apple.TCC"' --info --timeout 20s | egrep -i 'Auth|Denied|Allowed|bash|rsync|ditto|find|desktopwatcher'`

Watcher Script

```
#!/usr/bin/env bash
set -Eeuo pipefail
export LANG=C LC_ALL=C

DESKTOP="${DESKTOP:-$HOME/Desktop}"
INBOX_BASE="$HOME/.qai_optimize/inbox"
LOG_SYNC="$HOME/.qai_optimize/desktopwatcher.sync.log"

mkdir -p "$INBOX_BASE" "$(dirname "$LOG_SYNC")"

if ! /usr/bin/find "$DESKTOP" -mindepth 1 -maxdepth 1 -print -quit
    exit 0
fi

TS="$(/bin/date +%Y%m%d_%H%M%S)"
DST="$INBOX_BASE/$TS"
mkdir -p "$DST"

case "$DESKTOP" in
    */Users/*/Desktop) : ;;
    *) echo "[SAFEGUARD] Unexpected DESKTOP=$DESKTOP" && exit 1;;
esac

/usr/bin/ditto "$DESKTOP" "$DST" 2>>"$LOG_SYNC" || true
/usr/bin/rsync -a --remove-source-files --exclude ".DS_Store" "$DES

/usr/bin/find "$DESKTOP" -type d -mindepth 1 -prune -o -type f -pri
```

```
printf "[%s] synced -> %s\n" "$(/bin/date -Is)" "$DST" >>"$LOG_SYNC"
exit 0
```

13. Tek Seferlik Kurulum Bloğu

LaunchAgent, watcher script, .env dosyaları ve log dizinlerini idempotent biçimde oluşturan otomasyon komut bloğu.

```
set -Eeuo pipefail

PL="$HOME/Library/LaunchAgents/com.qai.desktopwatcher.plist"
SENT="$HOME/.qai_optimize"
BIN_DIR="$SENT/bin"
LOG_OUT="$SENT/desktopwatcher.out.log"
LOG_ERR="$SENT/desktopwatcher.err.log"
WATCHER="$BIN_DIR/qai_desktop_watcher.sh"
DESKTOP="$HOME/Desktop"

QAI_BASE="$HOME/QuantumAI"
mkdir -p "$QAI_BASE"
USDT_ABI_PATH="$QAI_BASE/USDT.json"

ENV_LOCAL="/Volumes/LaCie/Container-QuantumAI/QuantumAI-Dockerized-"
mkdir -p "$(dirname "$ENV_LOCAL")"
touch "$ENV_LOCAL"

mkdir -p "$BIN_DIR" "$SENT"
: > "$LOG_OUT"
: > "$LOG_ERR"

cat > "$WATCHER" <<'QAI_WATCHER'
#!/usr/bin/env bash
set -Eeuo pipefail
export LANG=C LC_ALL=C

DESKTOP="${DESKTOP:-$HOME/Desktop}"
INBOX_BASE="$HOME/.qai_optimize/inbox"
LOG_SYNC="$HOME/.qai_optimize/desktopwatcher.sync.log"
```

```

mkdir -p "$INBOX_BASE" "$(dirname "$LOG_SYNC")"

if ! /usr/bin/find "$DESKTOP" -mindepth 1 -maxdepth 1 -print -quit
    exit 0
fi

TS="$(/bin/date +%Y%m%d_%H%M%S)"
DST="$INBOX_BASE/$TS"
mkdir -p "$DST"

case "$DESKTOP" in
    */Users/*/Desktop) : ;;
    *) echo "[SAFEGUARD] Unexpected DESKTOP=$DESKTOP" >&2; exit 1;;
esac

/usr/bin/ditto "$DESKTOP" "$DST" 2>>"$LOG_SYNC" || true
/usr/bin/rsync -a --remove-source-files --exclude ".DS_Store" "$DES

/usr/bin/find "$DESKTOP" -type d -mindepth 1 -prune -o -type f -pri

printf '[%s] synced -> %s\n' "$(/bin/date -Is)" "$DST" >>"$LOG_SYNC
exit 0
QAI_WATCHER

chmod 755 "$WATCHER"

mkdir -p "$HOME/Library/LaunchAgents"
/usr/bin/printf '%s\n' \
'' \
'' \
'' \
' Labelcom.qai.desktopwatcher' \
' ProgramArguments' \
' /bin/bash' \
' -lc' \
" $WATCHER" \
' ' \
" WatchPaths$DESKTOP" \
' RunAtLoad' \
' KeepAlive' \
' StandardOutPath'"$LOG_OUT"' \
' StandardErrorPath'"$LOG_ERR"' \
'' > "$PL"

```

```
launchctl unload "$PL" 2>/dev/null || true
launchctl load "$PL"

printf 'Kurulum tamamlandi. Watcher: %s\n' "$WATCHER"
```

14. API & Entegrasyon

REST Uç Noktaları

```
POST /api/v1/orders
GET  /api/v1/orders/{orderId}
GET  /api/v1/orders/status

GET  /api/v1/risk/limits
POST /api/v1/risk/assessment

GET  /api/v1/markets/{symbol}/book
GET  /api/v1/markets/{symbol}/trades

POST /api/v1/grid/create
GET  /api/v1/grid/{gridId}/status
```

WebSocket Kanalları

```
/ws/v1/market/{symbol}:
- orderbook.update
- trades.update
- ticker.update

/ws/v1/orders:
- order.created
- order.updated
- order.filled
- order.cancelled
```

```
/ws/v1/system:
- health.update
- metrics.update
```

API Doğrulaması

```
import hmac
import time
import base64

def generate_signature(api_secret: str, timestamp: str, method: str,
                       message: str) -> str:
    message = f"{timestamp}{method}{endpoint}{body}"
    signature = hmac.new(api_secret.encode('utf-8'), message.encode('utf-8'),
                          hashlib.sha256).digest()
    return base64.b64encode(signature).decode('utf-8')

def create_auth_headers(api_key: str, api_secret: str, method: str,
                        timestamp: str) -> dict:
    signature = generate_signature(api_secret, timestamp, method,
                                   f"{timestamp}{method}{endpoint}{body}")
    return {
        'QAI-API-KEY': api_key,
        'QAI-TIMESTAMP': timestamp,
        'QAI-SIGNATURE': signature
    }
```

Oran Sınırları

```
rate_limits:
  public_endpoints:
    requests_per_minute: 60
    burst_size: 100
  private_endpoints:
    orders:
      requests_per_minute: 300
      burst_size: 500
    market_data:
      requests_per_minute: 600
      burst_size: 1000
```


Çevre Yapılandırması

```
COMPOSE_FILE="$ROOT/docker-compose.yml"
```

```
COMPOSE_PROJECT_NAME="qai"
```

```
BINANCE_TESTNET="1"
```

```
LOG_LEVEL="INFO"
```

15. Destek & Kaynaklar

Dokümantasyon

- [/docs/components/](#) → Bileşen belgeleri
- [/docs/api/](#) → API açıklamaları
- [/docs/config/](#) → Konfigürasyon rehberi
- [/docs/guides/](#) → Kullanıcı rehberleri

İletişim

- Teknik Destek: support@quantum-ai.com
- Acil Durum: emergency@quantum-ai.com
- Dokümantasyon: docs@quantum-ai.com

Referans Bağlantılar

- GitHub: <https://github.com/quantum-ai/trading-system>
- Dokümantasyon Portalı: <https://docs.quantum-ai.com>
- Durum Sayfası: <https://status.quantum-ai.com>

16. Mega Master Boru Hatları 360°

QuantumAI Mega Master Boru Hatları, küçük tutarlı perakende işlemlerden milyar seviyesindeki kurumsal hacimlere kadar tüm akışı tek bir düzen içinde toplar. Aşağıdaki maddeler, kullanıcı tarafından eklenen mega veri hattı senaryolarını operasyonel mimariye yerleştirir.

Veri Akışı Katmanları

- **Veri Girişi (Ingestion):**

Kafka/Redpanda üzerinden saniyede yüzlerce mikro emir ile milyon dolarlık işlemler aynı kuyruğa düşer.

- **Dinamik Yönlendirme:**

İşlemler içeriklerine göre küçük/büyük kollara ayrılır; >100K USD hacimler compliance kuyruğunda ek denetime tabi tutulur.

- **Paralel İşleme:**

Aynı olay risk motoru, finansal kayıt ve anomali tespiti için çoğaltılır; idempotency anahtarları çift işlemeyi engeller.

- **Çıktı & Depolama:**

UI güncellemeleri, rapor veri depolama ve AI eğitim setleri transaction-id tabanlı olarak senkron yürütülür.

Yük Senaryosu Kılavuzu

- **10 USD Mikro İşlem:**

Hızlı eşleşme, tek shard, retry limitleri ile saniyede 500+ emir.

- **1M USD Kurumsal:**

Çoklu imza doğrulaması, likidite kontrolü, düzenleyici iz kayıtları ve eş zamanlı risk simülasyonu.

- **AI Eşlik Akışı:**

Fiyat kayması ve trend analizi için gerçek zamanlı feature stream.

- **Uyumluluk:**

Tüm işlemler audit trail ve SLA metrikleri ile birlikte loglanır.

flowchart LR

```
Ingest[Veri Girişi] --> Classify{İşlem Sınıflandırma}
Classify -->|<100K USD| FastLane
Classify -->|>=100K USD| Compliance
FastLane --> TradeExec[Trade Engine]
Compliance --> RiskDesk[Risk Kontrol & Onay]
RiskDesk --> TradeExec
TradeExec --> Ledger[Finansal Kayıt]
TradeExec --> AIStream[AI Feature Store]
TradeExec --> Notify[UI / Bildirim]
Ledger --> Archive[(Uzun Süreli Arşiv)]
```

Mega İşlem Yol Haritası

Aşama	Küçük İşlem (10 USD)	Mega İşlem (1M USD)
Kimlik Doğrulama	Standart API anahtarı	MFA + imza zinciri, uyumluluk onayı
Risk Analizi	Hızlı risk oranı	Anlık VaR, limit kontrolü, ek raporlama
Yürütme	Tek borsa, mikro gecikme	Likidite bölme, TWAP/VWAP, slippage tahmini
Kaydetme	Redis + Postgres kayıt	Çok bölgeli replika, tam audit, reg raporu

17. Quantum AI Bot Programları

Kullanıcı tarafından eklenen eğitim modülleri, Mega Master pipeline üzerinde çalışan bot ekosistemini genişletir. Aşağıdaki içerik başlıkları aynı metinde yer alan talimatların yeniden düzenlenmiş halidir.

Bot Eğitim Dizisi

Balina & Piyasa Analizi

- Quantum AI Bot Kurulumu
- Quantum AI Bot Cüzdan & Güvenlik Ayarları
- Quantum AI God Mode Launch + Ön Satış Konfigürasyonu
- Copy Trade Kanalları ve ileri seviye öğrenme kaynakları
- Whale Alert, Nansen, Glassnode entegrasyonu ile gerçek zamanlı balina takibi.
- On-chain veri akışı için Etherscan, BSCScan, TronScan API kullanım senaryoları.
- AI destekli balina skorlaması (kârlılık, volatilité, drawdown, artan bakiyeler).

Copy Trade & God Mode Özet Akışı

- **API & Cüzdan Entegrasyonu:** Binance/OKX copy trade API'leri, zincir üstü adres izleme, mempool dinleme.
- **Gerçek Zamanlı Senkronizasyon:** Milisaniye gecikmeli emir klonlama, WebSocket tetikleyicileri, otomatik pozisyon boyutlama.
- **Risk & Uyumluluk:** Maksimum kayıp limitleri, kullanıcı bazlı filtreler, yönetilebilir stop-loss/TP.
- **AI Destekli Öneriler:** En yüksek Sharpe değerli trader listeleri, kişiselleştirilmiş God Mode bildirimleri.

18. Otomasyon & Sertifika Scriptleri

Eklenen otomasyon betiği, sertifika gömülmesi, AWS Bedrock entegrasyonu, Prometheus/Grafana manifestleri ve Python ortam hazırlığını aynı orkestrasyon altında birleştirir.

```
#!/usr/bin/env bash
```

```
print_banner() {  
cat << 'EOF'
```

```

/ _ \      | | ( )      | | | ( )      | |
| | | | _ _ _ _ _ _ | | _ _ _ _ | | | | _ _ _ _ | | _ _ _ _
| | | | | | | | / _ \ / _ \ | | | | / _ \ | | | | / _ \ | | / _ \ ' _ \
| | _ | | | _ | | _ \ | | | | ( | | | _ | | | | ( | | | < _ / |
\ _ _ \ _ \ _ , _ | \ _ _ | | _ _ \ _ | | \ _ , _ | \ _ \ _ | |

```

```

EOF
echo -e "${NC}"
}

secure_embed_and_bedrock() {
    log_info "[SECURE] Embedding certs/profiles and installing Bedr
    local ROOT_DIR="${QAI_BASE_PATH}"
    local SEC_DIR="${ROOT_DIR}/secrets"
    local LOG_DIR="${HOME}/qai_logs"
    local TS="$(date +%Y%m%d-%H%M%S)"
    mkdir -p "${SEC_DIR}" "${LOG_DIR}" || true
    umask 077
    local APPLE_CERT_DEV="${APPLE_CERT_DEV:-/mnt/data/development.c
    local APPLE_CERT_APPID="${APPLE_CERT_APPID:-/mnt/data/developer
    local APPLE_PROV_PROFILE="${APPLE_PROV_PROFILE:-/mnt/data/Quant
    if [ -f "${APPLE_CERT_DEV}" ]; then install -m 600 "${APPLE_CER
    if [ -f "${APPLE_CERT_APPID}" ]; then install -m 600 "${APPLE_C
    if [ -f "${APPLE_PROV_PROFILE}" ]; then install -m 600 "${APPLE
    if [ -d "${SEC_DIR}" ] && [ ! -f "${ROOT_DIR}/secrets.enc" ]; t
        ( cd "${ROOT_DIR}" && tar -cf - secrets | openssl enc -aes-
    fi
    if command -v security >/dev/null 2>&1; then
        [ -f "${SEC_DIR}/development.cer" ] && security add-trusted
        [ -f "${SEC_DIR}/developerID_application.cer" ] && security
    fi
    local PP_DST="${HOME}/Library/MobileDevice/Provisioning Profile
    mkdir -p "${PP_DST}" || true
    [ -f "${SEC_DIR}/Quantum_AI_Digital_Money_System_Panel.provisio
    # ... tam betik ham ek icerikte detayli olarak devam eder ...
}

provision_metrics_manifests() {
    log_info "[METRICS] Ensuring Prometheus and Grafana manifests e
    mkdir -p conf grafana/provisioning/datasources grafana/provisio
    if [ ! -f conf/prometheus.yml ]; then
        cat > conf/prometheus.yml <<'YML'
global: { scrape_interval: 5s }

```

```

scrape_configs:
  - job_name: 'gai_services'
    static_configs:
      - targets: ['api:9101','router:9102','risk:9103','small_agg:9
  - job_name: 'prometheus'
    static_configs: [{targets: ['prometheus:9090']}]}

YML
fi
# devamı ham ek içerikte yer almaktadır
}

setup_python_env() {
  log_info "Setting up Python environment"
  pip install docker redis aiohttp websockets prometheus-client p
}

main() {
  case "$1" in
    start)
      setup_colima
      secure_embed_and_bedrock
      provision_metrics_manifests
      ;;
    # diğer durumlar...
  esac
}

```

Komple betik içeriği, aşağıdaki ham ek veri arşivinde referans olarak saklanır; böylece hiçbir komut kaybolmaz.

19. Genişletilmiş Bilgi Notları

Ek notlar; kullanıcı tarafından sağlanan kuantum kriptografi, QKD/QRNG, tokenizasyon ve regülasyon vurgularını modüler başlıklar halinde sunar.

- **Kuantum AI & Kriptografi:** Shor ve Grover algoritmalarının RSA/AES etkileri, kuantum sinir ağlarıyla hızlandırılmış anahtar kurtarma, çift taraflı stratejiler.
- **QKD & QRNG Entegrasyonu:** BB84 protokolü, kuantum hata düzeltme, 5G

finans ağlarında hibrit kullanım.

- **Tokenizasyon Motoru:** ERC-20/TRC-20 sözleşme üretimi, OpenZeppelin şablonları, holder/transfer izleme, çoklu zincir gaz planlaması.
- **PCI & Regülasyon:** TLS 1.3, sertifika pinning, script bütünlüğü, SIEM alarm hatları ve pen-test gereksinimleri.
- **Stratejik Yol Haritası:** Sprint bazlı mega pipeline, FinOps ısı haritaları, Crossplane kompozisyonları ve cloud-native gözlemlenebilirlik.

0xFF. Ham Ek İçerik Arşivi

Bu bölüm, kullanıcı tarafından sağlanan tüm ham metin, kod ve notların eksiksiz saklanması için eklenmiştir. Profesyonel sürümde özetlenen her bilgi burada orijinal biçimiyle korunur.

 Ham içeriği göster (6621 satır)

QuantumAI © 2025 • Saha operasyon ekipleri için hazırlanmıştır. Belge, %99.95 hata bütçesi hedefiyle otomatik olarak güncellenir.

Notlar (Docker Context)

- Komutla bağlam sabitle: `docker context use colima-qai`